

StudyPilot

Group 3

Monabbir Bhuiyan

Tessa Cloutier

Gabriella Crook

Ashvin Kawleswaran

Sandeepon Saha

Activity 1 Tester Assignment

Group Pilot – Implemented User Stories and Testing Overview

The Group Pilot system is designed to support professors in forming balanced and effective student groups using survey-driven data. Based on the current implementation, several core user stories have been fully developed, tested and integrated with a GUI. These user stories span authentication, course management, group formation, availability tracking and analytics, in order to form a cohesive and functional application.

One of the foundational user stories implemented is user authentication. As a user, individuals can register and log into the system to access its features. This functionality is supported by the `AuthenticationController`, along with `LoginForm` and `RegisterForm`, and corresponding GUI pages (`login.html` and `register.html`). These components have been tested to ensure proper validation, data handling, and user session control.

Another key feature is the landing page and navigation system, which allows users to access different parts of the application. The `LandingController` and `landing-page.html` provide an entry point into the system, ensuring a smooth user experience. This has been tested for correct routing and accessibility.

From the professor's perspective, course management is fully implemented. Professors can view and manage their courses through the `ProfessorHomeController` and associated pages such as `professor_home.html`, `professor_course.html`, and `professor_course_page.html`. Integration testing ensures that course data is properly retrieved and displayed.

For students, the system supports course enrollment, allowing them to join courses and participate in group activities. This is handled by `StudentJoinCourseForm`, `StudentHomeController` and pages like `student_course_page.html` and `my_courses.html`. Unit and integration tests verify that students can successfully enroll and view their courses.

A central feature of the application is group formation setup. Professors can define grouping criteria using the `GroupFormationController` and `CreateGroupFormationForm`, with GUI pages such as `group_formation_page.html` and `group_formation_edit_page.html`. This functionality has been tested to ensure that input data is correctly processed and stored for group generation.

Students contribute to this process through group preference submission, where they provide input on their skills, interests and preferences. This is implemented via `StudentGroupPreferenceForm` and connected backend logic. Testing ensures that submissions are correctly recorded and associated with the appropriate student profiles.

In addition, the system incorporates group availability collection, which supports future meeting scheduling. As a student, users can submit their availability, allowing the system to determine overlapping time slots within a group. A full scheduling interface for professors is implemented as

well. This ensures that availability data can later be used to automate or assist in scheduling group meetings. This feature has been tested to confirm that availability inputs are captured and stored correctly.

The system also includes a team health monitoring feature, where students complete weekly surveys to assess group dynamics. This is managed by the `TeamHealthController` and `TeamHealthCheckin` model. Integration testing confirms that survey responses are stored and can be used for further analysis.

An analytics dashboard is available through `analytics.html`, allowing professors to view summarized data about teams. Additional supporting features include a FAQ/help page (`faq_library.html`) and a settings page (`settings.html`), which enhance usability and personalization. These interfaces have been tested for accessibility and navigation.

Overall, the implemented user stories demonstrate a strong integration between backend logic, frontend GUI, and testing practices. Unit tests validate individual components such as forms and models, while integration tests ensure that controllers, services and views work together. These completed features collectively support the primary goal of Group Pilot: enabling efficient, data-driven group formation for improved collaboration and outcomes for students and professors.

Story Testing Responsibilities

Tessa — testing:

- User can register an account
- Professor can create a course
- Student can join a course with a join code

Monabbir — testing:

- Professor can create group formations
- Student can set group preferences
- Professor can view enrolled students in a course

Ashvin — testing:

- User can log in
- The professor can publish a weekly survey
- Students can submit a weekly health check-in

Gabriella —testing

- Teams can set availability
- Prof can access team meeting times and overlaps in availability